Which LLM Wrote This?

Rhianna Ferreira (team leader, solo) · Oct 12, 2026

Overview

In this project I trained a CNN and an LSTM to identify which large language model wrote a given answer. On five models from the TXD-22 dataset which included data from ChatGPT, Claude, Gemini, Llama and DeepSeek, the CNN gets 88.3% test accuracy and the LSTM gets 74.6%. Against random guessing, it would get 20.0%.

1 Problem Definition and Dataset Curation

1.1 Task

We are an answer y written by a language model, and the goal is to create and train a model to predict which model wrote it. This is a complex problem as every model is a chatbot trained on similar data for similar goals, which causes similarities throughout.

1.2 Datasets

I used two public datasets. TXD-22 is used for RQ1, RQ2 and RQ4. M4 is used only for RQ3, as it labels each example with a topic area and TXD-22 does not.

TXD-22 (main dataset). *TXD-22: A Large-Scale Benchmark Dataset for Multi-Class AI-Generated and Mixed-Source Text Identification*, Mendeley Data V1 (July 2026), doi.org/10.17632/prcjcggtjf.1.

Its authors asked the same questions to many LLMs with answers capped near 300 words. The release has 66,000 question-answer pairs over 22 classes with 3,000 per class.

Its columns map directly onto what the assignment asks for as Question is the LLM input, Answer is the LLM output and Source is the LLM name. I kept the five single-model classes ChatGPT, Claude, Gemini, Llama and DeepSeek, with 2,999 examples each. It is 14,995 in total as one Llama answer was dropped for being too short to allow the classes to be balanced. The raw CSV is over GitHub's 100 MB file-size limit, so it is not in the repository. The CSV is mostly UTF-8 but it also contains curly quotes, dashes, ellipses. Reading the whole file as `latin-1` would reproduce every real UTF-8 character, and punctuation is one of the things RQ4 looks at,. The build script fixes the bad bytes one at a time and leaves the rest alone.

M4 (for RQ3). Wang et al. (EACL 2024), *M4: Multi-generator, Multi-domain, and Multi-lingual Black-Box Machine-Generated Text Detection*, publicly available on GitHub.

I took 20,000 examples which consist of five models (ChatGPT, text-davinci-003, Cohere, Dolly-v2, BLOOMz) times four domains (Wikipedia, WikiHow, Reddit ELI5, arXiv), with there being 1,000 per model per domain. I skipped M4's fifth domain (PeerRead) as its files use a different format that does not split cleanly into one prompt and one answer.

1.3 Preprocessing

Normal text cleaning like lowercasing, removing punctuation and markdown would not be helpful as we lose the style clues of the LLMs that are needed to help with classification, so I kept punctuation, markdown symbols and line breaks as their own tokens and only lowercased words. The vocabulary is built from the training set only up to 30,000 words that appear at least twice. The data is split 80/10/10 into train, validation and test, keeping the same class mix in each part.

2 Model and Training Details

2.1 Models

Both models are written in PyTorch and learn their own word embeddings with no pretrained weights used.

CNN: Word embeddings (d = 128) go through four 1-D convolutions with window sizes of 2, 3, 4 and 5 (100 filters each), then ReLU, max-pooling over the whole text, dropout (p = 0.5) and a final linear layer. Max-pooling helps to let one telling phrase anywhere in the answer be able to guide the prediction.

LSTM (the RNN). Word embeddings (d = 128) go through a bidirectional LSTM. Instead of only using the last hidden state, I take the mean and the max over all time steps so that the start of a long answer is not just forgotten and then dropout and a linear layer. Padding is skipped with packed sequences and gradients are clipped at 5.

2.2 Training setup

Setting	CNN	LSTM
Parameters	3,895,781	3,889,669
Max sequence length (tokens)	256	128
Hidden size	4 × 100 filters	96 (each direction)
Batch size	64	64
Optimiser	Adam	Adam
Learning rate	0.001	0.001
Weight decay	0.00001	0.00001
Dropout	0.5	0.5
Max epochs	6	6

Both models use cross-entropy loss and I keep the epoch with the best validation macro-F1 and stop early if it does not improve for two epochs. I used macro-F1 rather than accuracy so that one class doing bad is not able to hide behind the others. Random seeds are fixed so the runs can be repeated.

The split gives 11,995 training, 1,500 validation and 1,500 test answers, and the vocabulary has 29,017 words. For the prompt and answer runs, the prompt and the answer are joined with a sep token so the model can tell the difference and training on the answers took about 14 minutes for the CNN and about 6 minutes for the LSTM.

Compute limits. Everything ran on a CPU with 2 cores and no GPU and at 256 tokens one LSTM epoch took about 18 minutes, which was too slow for all the experiments, so the LSTM reads 128 tokens with 96 hidden units. Both models see the same data and splits, but this difference means the CNN-vs-LSTM comparison is not perfectly fair.

3 Results

Note: the figures in this report were made with code written with the help of an LLM.

3.1 RQ1: Can we identify which LLM generated a response?

Yes, you can identify which LLM generated the answer alone, the CNN is right 88.3% of the time, about 4.4 times better than guessing, and the LSTM gets 74.6%. Every class is recognised well: CNN F1 ranges from 0.815 (DeepSeek) to 0.962 (Llama).

Input	Model	Accuracy	Macro-F1	Best epoch
Answer only	CNN	0.8833	0.8827	6
Answer only	LSTM	0.7460	0.7473	5
Prompt only	CNN	0.1147	0.0879	2
Prompt only	LSTM	0.1347	0.1056	2
Prompt + answer	CNN	0.8800	0.8798	6
Prompt + answer	LSTM	0.7480	0.7463	5

TXD-22, five classes, random split; guessing gives 20%.

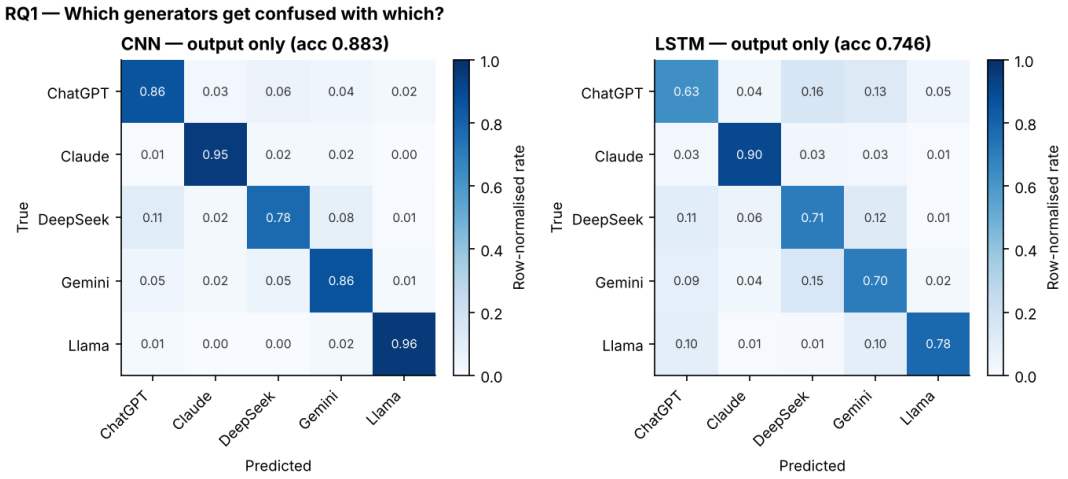


Figure 1: Confusion matrices (answer only), normalised by row. Off-diagonal cells show which models get mixed up.

3.2 RQ2: Does the user's prompt help?

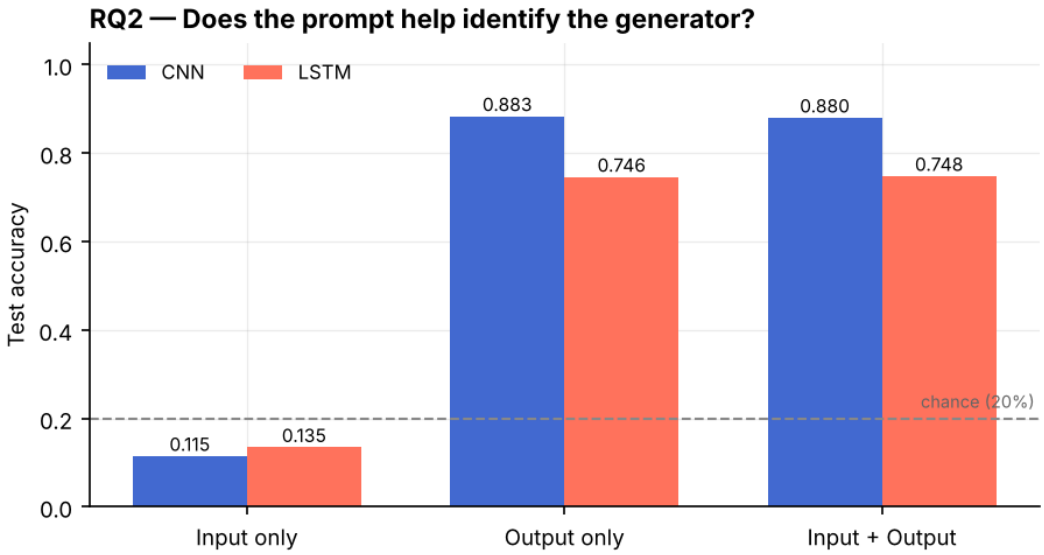


Figure 2: Accuracy for each kind of input. Prompt-only is at or below guessing.

(i) Can the LLM be predicted from the prompt alone? No, the LLM cannot be predicted from the prompt alone. Because with the corrected split the CNN gets 20.0% which is the similar

to what you would get if you guessed randomly. That makes sense: every model in TXD-22 was asked the same questions, so the question says nothing about who answered. My first run gave 11.5%, which is below the accuracy when you guess. That turned out to be a mistake in my data split.

(ii) Does adding the prompt help? No. With the corrected split, the CNN gets 84.9% with prompt and answer and 86.3% with the answer alone which is -1.4 points. The prompt adds no useful information and takes up part of the 256 token limit.

(iii) If prompt-only classification worked surprisingly well, why? It did not work well here. It could have in a dataset where each model was given its own set of prompts, because the classifier would then learn the prompts rather than the models. I checked for this by taking the first 12 words of each prompt and counting how many appear for only one model and between 0.0% and 0.2% per model, so TXD-22 does not have this problem.

3.3 RQ3: Do fingerprints generalise across domains? (extra credit)

For each of M4's four domains, I trained on the other three and tested on the one left out to keep the test topic completely new to the model.

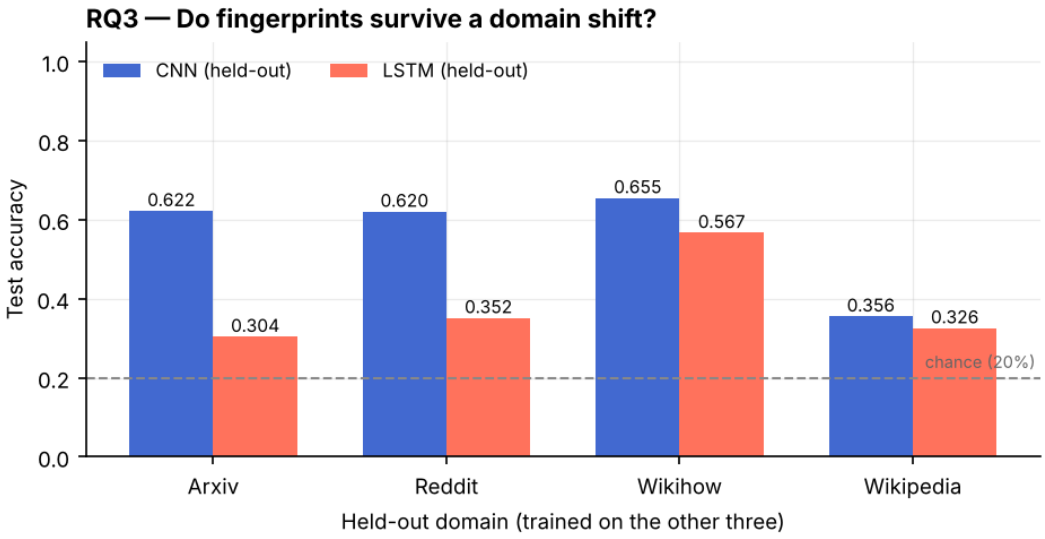


Figure 3: Test accuracy on each held-out domain (trained on the other three).

(i) Does performance drop on unseen domains? Yes, this is seen to occur as the CNN averages 56.3% on the held-out domain against 84.0% on validation data from the domains it had trained on. The LSTM averages 38.7% on the held-out domain against 78.8% on the domains it trained on. Wikipedia is the hardest domain to transfer to with the CNN at 35.6%, though all the results stay above the 20% guessing rate.

(ii) Are some models identifiable more consistently across domains? Yes. BLOOMz is the most consistent as its CNN F1 never falls below 0.78. ChatGPT varied the most from 0.82 down to 0.00 depending on which domain is held out. Overall, some of the models have a style that is able to be carried across topics, while others are only recognisable on topics the classifier has seen.

Model	arXiv	Reddit	WikiHow	Wikipedia
BLOOMz	0.99	0.98	1.00	0.78

Model	arXiv	Reddit	WikiHow	Wikipedia
Dolly-v2	0.54	0.70	0.93	0.40
Cohere	0.54	0.46	0.50	0.21
ChatGPT	0.82	0.32	0.20	0.00
text-davinci-003	0.13	0.56	0.55	0.08

(iii) General fingerprint or topic-specific patterns? I would say both as accuracy stays above guessing on all new domains, so part of what the classifier learns is considered a general fingerprint. The large drop compared with the seen domains clarifies that a good part of it is tied to the topics that it is trained on.

3.4 RQ4: What characteristics distinguish the models? (extra credit)

To see what aspects differs between the models, I computed 56 simple statistics for each answer: length, sentence length, vocabulary variety, formatting, punctuation rates and how frequency of common words. A random forest trained on these alone gets 85.7%, which is close to the CNN's 88.3%. The ten most useful features are:

Feature	Importance
`mean_sentence_len`	0.0865
`entropy`	0.0846
`root_ttr`	0.0721
`n_chars`	0.0707
`n_sentences`	0.0619
`n_words`	0.0611
`ttr`	0.0601
`dash_per_100w`	0.0547
`fw_and`	0.0341
`uppercase_ratio`	0.0307

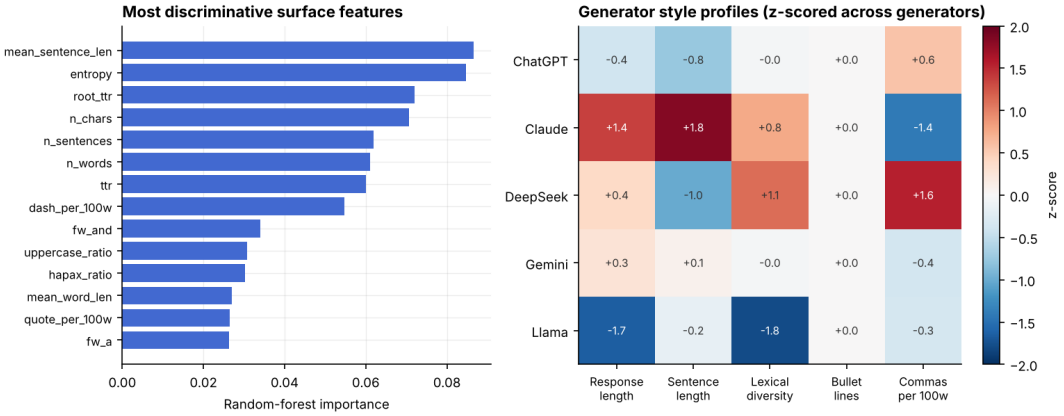


Figure 4: Left: random-forest feature importance. Right: each model's average feature values, standardised across models.

The top features mix structure including sentence length, number of characters, words and sentences with language features of word entropy, vocabulary variety, how often "and" appears, and punctuation.

To see which signals the neural models actually rely on, I removed one kind of signal at a time and retrained from scratch.

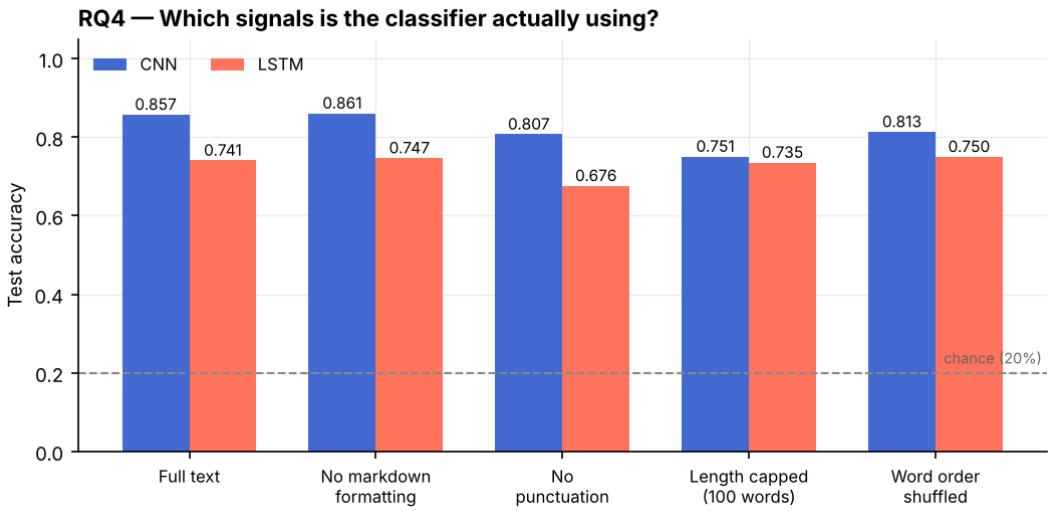


Figure 5: Accuracy after removing each kind of signal.

These runs use 4 epochs and it was 85.7% for the CNN, 74.1% for the LSTM and that is the baseline to compare against. Cutting every answer to 100 words hurts the CNN most was -10.7 points, then removing punctuation was -5.0 and shuffling word order was -4.4. Removing markdown changes almost nothing just +0.3, because TXD-22 answers are mostly plain prose with little formatting. Even with any one signal removed, accuracy stays far above guessing, so no single signal explains the result. Each run used one seed, so differences of a point or two may be noise.

4 In-Depth Analysis

4.1 A below-chance result that exposed a mistake in my data split

In my first run, the prompt-only CNN scored 11.5%, well below the 20% you get by guessing. Guessing cannot do that badly, so something was systematically wrong.

The cause was the split. In TXD-22 each question is answered by all five models and 2,957 of 3,014 questions appear exactly five times. With a random split, when a question lands in the test set, its four other copies are usually in the training set with the four *other* labels. The model learns that this question belongs to those four models and so picks any model except the right one.

To fix it, I re-ran everything with a grouped split where all five answers to a question go into the same part and I checked that no question appears in both train and test.

Input	Model	Random split	Grouped split
Prompt only	CNN	0.1147	0.2000
Prompt only	LSTM	0.1347	0.2007
Answer only	CNN	0.8833	0.8633
Answer only	LSTM	0.7460	0.7515
Prompt + answer	CNN	0.8800	0.8492
Prompt + answer	LSTM	0.7480	0.7253

With the grouped split, prompt-only accuracy becomes 20.0%, exactly chance, which is the correct answer for this dataset. Answer accuracy barely moves from 88.3% to 86.3%, -2.0 points, so the RQ1 result did not depend on the mistake.

4.2 Which models get confused with each other?

The CNN's most common mistakes are DeepSeek taken for ChatGPT was 11.3% of DeepSeek answers, DeepSeek taken for Gemini was 8.3% and ChatGPT taken for DeepSeek was 5.7% of ChatGPT answers. The mistakes are not symmetric: a model with a distinctive style is rarely mistaken for a plainer one, but the reverse happens more often.

4.3 Limitations

- The CNN reads 256 tokens and the LSTM 128, because of the CPU limit, so this project does not show which architecture is better in general.
- Each setting was run with one random seed, so differences of 1 to 2 points should not be over-interpreted.
- TXD-22 answers are capped near 300 words, which limits how much answer length can vary.
- Both datasets capture specific model versions. Chatbots are updated all the time, so a classifier trained today may not work on next year's models.

5 Lessons and Experience

- I expected building and training the models to be the main part, but cleaning the data in order to make it right took longer. The TXD-22 file had broken characters that had to be fixed one byte at a time, while I had to use the M4's files, and one used a different format, so I had to drop that domain. I understand the importance and time needed to clean the data in order to make the models more accurate as the smallest aspects can make an impact.
- With only a 2-core CPU, the LSTM was too slow at 256 tokens, so I had to cut it to 128 tokens, which made the comparison unfair. I think if I did it again, I would use a GPU, or give both models the same shorter length so that the comparison is fair.
- From the signals part, it helped me see that the feature statistics made it look like a few things were doing most of the work, but when I removed any one of them, the accuracy above guessing. This taught me how the models differ in many small ways all at once and that a feature looking important is not the same as the model depending on it.